Literature Survey and Research Gap Analysis: AI Data Leakage Prevention & Detection System

Project Review 2 submission

1. Introduction

Large Language Models (LLMs) are now used for daily enterprise work such as debugging, drafting, and analysis. Employees paste API keys, customer PII, and proprietary details into prompts, and LLM agents can leak secrets through tools, shared context, or inference. Classical Data Loss Prevention (DLP) was built for structured identifiers (credit cards, SSNs) and struggles with free-form prompts. This survey reviews recent work on leakage prevention and detection for LLM systems, compares the methods, and identifies the gaps that our project targets.

Papers [1]–[5] were read in full or in their key sections (abstract, method, results, limitations). Papers [6]–[10] are foundational works cited by [1] and should be read before final submission (see the note in Section 6).

2. Summary of Reviewed Papers

[1] SafeGPT (Desai et al., 2026). A two-sided guardrail for enterprise LLM use. The input side uses regex pattern matching, contextual NER, and knowledge-graph similarity, with graduated actions (block, warn, redact). The output side uses bias, policy, and factual-consistency classifiers with automatic reframing, plus a human-in-the-loop feedback loop. Evaluated on three synthetic datasets (PIIBench, ToxicChat, EnterpriseScenarios). Ablation shows pattern matching contributes most on PIIBench (recall drops 55 points without it).

[2] Instructional Text-induced Leakage (Kao et al., 2026). Shows that high-privilege LLM agents follow instructions hidden in project README files, causing data exfiltration. Introduces ReadSecBench (500 real-world READMEs). End-to-end exfiltration succeeds up to 85%. In a 15-person user study, no participant detected the malicious instruction (0%), and 12 rule-based and 6 LLM-based defenses did not detect reliably without high false positives.

[3] PRISM (Tapwal et al., 2026). Detects secret leakage in multi-agent LLM pipelines during generation. Combines 16 signals (lexical, structural, information-theoretic, behavioural, contextual) into a per-token risk score with green/yellow/red zones. On a

2,000-task benchmark: F1 = 0.832, precision = 1.000, recall = 0.712, 0.0% task-level leak rate, utility 0.893. The undefended pipeline leaked secrets in 71.2% of tasks.

[4] Multi-Agent Firewall (García Cuesta et al., 2026). Open-source, local-first user-facing firewall. A browser extension plus a MitM proxy intercept HTTP(S) and WebSocket traffic. A multi-agent pipeline mixes deterministic detectors with LLM semantic analysis, OCR/VLM for files and screenshots, fuzzy matching against Git repositories for code leakage, and a risk-based policy engine. Reports F1 up to 94.93% in the best configuration.

[5] Agent Mosaic (Ruiz et al., 2026). Studies the mosaic effect: an agent combines individually permitted documents and discloses a forbidden conclusion. On 30 synthetic examples (10,800 simulated answers), the best model still discloses forbidden composites in 7.08% of answers on benign queries. Adding classification headers and aggregation rules lowers but does not remove disclosure (32.37% pooled). Refusals can themselves reveal the protected conclusion.

3. Literature Survey Comparison Table

Ref	Focus	Method	Where it acts	Dataset / evaluation	Key result	Limitation
[1] SafeGPT	Enterprise prompt and response safety	Regex + NER + knowledge graph; output classifiers; feedback loop	Input and output	3 synthetic datasets; 4 baselines	PIIBench recall 70.0%, precision 100%; EnterpriseScenarios precision 40.5%, FPR 78.6%	Synthetic data only; high over-blocking on enterprise data; no adversarial, latency, or deployment tests
[2] Kao et al.	Instruction-induced exfiltration by agents	Taxonomy + ReadSecBench measurement; user study; defense evaluation	Agent input (documentation)	500 real READMEs; 4 model families; 15 users	Up to 85% exfiltration; 0% human detection	Measures the attack; proposes no working defense
[3] PRISM	Secret leakage across agent hops	16-signal per-token risk score during decoding	Generation time	2,000 adversarial tasks, 13 attack types	F1 0.832, precision 1.0, recall 0.712, 0% leak	Recall 71.2%; needs token-level (logit) access; credential-focused; benchmark-specific
[4] MA Firewall	User-facing DLP for web and API LLM use	Browser extension + proxy; deterministic + LLM detectors; OCR/VLM; Git fuzzy match	Input (and traffic)	Reported F1 up to 94.93% (best config)	Local-first, covers files and code leakage	Best-case configuration; cost/latency trade-offs; output side is secondary

Ref	Focus	Method	Where it acts	Dataset / evaluation	Key result	Limitation
[5] Agent Mosaic	Inference and aggregation leakage	Controlled simulation + live agent harnesses	Agent answer	30 synthetic examples; 10,800 + 1,440 answers	7.08% leak (best model); 32.37% with safeguards	Synthetic, small scenario set; measures the problem, not a detector

4. Critical Observations

1. **Headline numbers and tables differ in [1].** The abstract and conclusion claim 92% precision and 87% recall, but Table 1 shows PIIBench recall of 70.0% and EnterpriseScenarios precision of 40.5%. The feedback gain (34% fewer false positives) and 4.0/5 satisfaction are also not backed by a table in the paper body. We treat these figures cautiously.
2. **Some baseline results look degenerate.** In [1], every baseline scores 0% on ToxicChat and Simple NER scores 0% on PIIBench, which suggests the baselines are not designed for those tasks rather than a fair comparison.
3. **Different papers protect different surfaces.** [1] and [4] protect user prompts, [2] and [5] study agents, and [3] protects multi-agent generation. No reviewed paper covers all of them.
4. **Datasets are mostly synthetic or custom.** Results are not directly comparable across papers.

5. Research Gaps Identified

#	Gap	Evidence
G1	**No realistic, shared benchmark for enterprise prompt-leakage detection.** Results rely on synthetic or custom datasets, so systems cannot be compared fairly.	[1] uses three synthetic datasets; [3] and [5] use their own benchmarks; [4] reports best-configuration scores.
G2	**Precision vs. recall trade-off is unresolved.** Context-aware detectors (knowledge graph) over-block benign content, which causes workflow disruption and alert fatigue.	[1] reports 78.6% FPR on EnterpriseScenarios; [4] notes alert fatigue.

#	Gap	Evidence
G3	**Detectors check what is written, not what is inferred.** Pattern, NER, and similarity checks cannot catch a forbidden conclusion built from permitted pieces.	[5]: leakage persists at 32.37% even with safeguards; refusals can leak.
G4	**Instruction-based leakage in agents is largely undetected.** Prompt-level filters do not inspect documents or tool content that agents obey, and humans and current defenses fail to notice.	[2]: 85% exfiltration, 0% human detection, 18 defenses unreliable.
G5	**Leakage propagation across agents and tools.** Defenses for single prompt-response pairs ignore shared context. The strongest generation-time detector needs model internals, so it cannot wrap closed API models (our inference from the method).	[3]: 71.2% undefended leak rate; recall only 71.2%.
G6	**No unified architecture across channels.** Browser prompts, API traffic, uploaded files, code, and agent actions are handled by separate systems.	[1], [4] input-centric; [2], [3], [5] agent-centric.
G7	**Missing real-world operational evaluation.** Latency at scale, adversarial bypass, user adaptation, and compliance mapping (GDPR, HIPAA, SOC 2) are not tested.	[1] Limitations section; [4] cost/latency trade-off.
G8	**Weak explainability and audit trail for detections.** Users are blocked or redacted without clear, auditable reasons tied to policy.	Partly addressed by logging in [1]; not evaluated in any paper.

6. How Our Proposed System Differs

Edit this section to match your actual design. The points below are a suggested positioning based on the gaps above.

The proposed **AI Data Leakage Prevention & Detection System** is a layered, channel-aware pipeline that monitors prompts, uploaded files, retrieved context, and model responses in one policy framework.

Aspect	Existing systems	Proposed system
Coverage	Single surface: prompt ([1], [4]) or agent ([2], [3], [5])	Prompt + file + retrieved-context + response in one pipeline (G6)
Detection	Pattern/NER/KG, or per-token signals needing model access	Pattern + NER + semantic classifier; model-agnostic (works with API LLMs) (G5)
False positives	High on enterprise data ([1]: 78.6% FPR)	Risk-score thresholds with redact-first policy and user feedback calibration (G2)
Inference leakage	Not handled by detectors	Sensitivity tagging and aggregation checks on the final response (G3)
Agent/document instructions	Undefended ([2])	Scan retrieved documents for embedded instructions before the model sees them (G4)
Evaluation	Synthetic, self-built	Mixed public + realistic test sets, reported with precision, recall, FPR, latency (G1, G7)
Auditability	Limited	Logged decision with reason and matched policy per detection (G8)

Note on the reading list: the rubric asks for 5–10 papers that you have actually read. Papers [1]–[5] above are fully covered. Papers [6]–[10] are standard references cited in [1]; read at least their abstracts and key results, and rewrite the one-line notes in your own words before submission.

7. References (IEEE Format)

[1] P. Desai, L. Tang, Y. Meng, and Z. Xi, "SafeGPT: Preventing data leakage and unethical outputs in enterprise LLM use," arXiv:2601.06366v3 [cs.CR], May 2026.

[2] C.-Y. Kao, X. Li, S. Dai, T. Qiu, P. Zhou, E. H. Jiang, and P. Sperl, "You told me to do it: Measuring instructional text-induced private data leakage in LLM agents," arXiv:2603.11862v1 [cs.CR], Mar. 2026.

[3] R. Tapwal, A. Kumar, and C. Maple, "PRISM: Generation-time detection and mitigation of secret leakage in multi-agent LLM pipelines," arXiv:2605.10614v1 [cs.AI], May 2026.

[4] H. García Cuesta, P. Mateo Torrejón, and A. Sánchez-Macián, "Multi-agent firewall architecture for privacy protection of sensitive data in interactions with language models," arXiv:2607.08282v1 [cs.CR], Jul. 2026.

[5] D. C. Ruiz, L. Luceri, and E. Ferrara, "Agent Mosaic: Recognizing confidentiality leaks in agentic LLM systems," arXiv:2610.30418v1 [cs.CR], Oct. 2026.

[6] N. Carlini et al., "Extracting training data from large language models," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2633–2650.

[7] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. EMNLP 2023: System Demonstrations*, 2023, pp. 431–445.

[8] T. Markov et al., "A holistic approach to undesired content detection in the real world," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 15009–15018.

[9] S. Liu and R. Kuhn, "Data loss prevention," *IT Professional*, vol. 12, no. 2, pp. 10–13, 2010.

[10] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, 2023.